Creating a Simple Gradle Project.

Step 1:Create a directory in Devops called MyGradleApp , go to the directory and execute the command gradle init –type java-application

```
vboxuser@niranjan: ~/devops/MyGradleapp
vboxuser@niranjan:~$ cd devops
vboxuser@niranjan:~/devops$ mkdir MyGradleapp
vboxuser@niranjan:~/devops$ cd MyGradleapp
vboxuser@niranjan:~/devops/MyGradleapp$ gradle init --type java-application
Starting a Gradle Daemon, 1 incompatible and 1 stopped Daemons could not be reus
ed, use --status for details

BUILD SUCCESSFUL in 12s
2 actionable tasks: 2 executed
vboxuser@niranjan:~/devops/MyGradleapp$
```

Step 2: Run **gradle build** command and display the tree using **tree** command

```
vboxuser@niranjan: ~/devops/MyGradleapp
vboxuser@niranjan:~/devops/MyGradleapp$ gradle build

BUILD SUCCESSFUL in 22s
7 actionable tasks: 7 executed
vboxuser@niranjan:~/devops/MyGradleapp$ ls
build  build.gradle  gradle  gradlew  gradlew.bat  settings.gradle  src
vboxuser@niranjan:~/devops/MyGradleapp$
```

```
vboxuser@niranjan: ~/devops/MyGradleapp
vboxuser@niranjan:~/devops/MyGradleapp$ tree
.
├── build
│   ├── classes
│   │   └── java
│   │       ├── main
│   │       │   └── App.class
│   │       └── test
│   │           └── AppTest.class
│   ├── distributions
│   │   ├── MyGradleapp.tar
│   │   └── MyGradleapp.zip
│   ├── libs
│   │   └── MyGradleapp.jar
│   ├── reports
│   │   └── tests
│   │       └── test
│   │           ├── classes
│   │           │   └── AppTest.html
│   │           ├── css
│   │           │   ├── base-style.css
│   │           │   └── style.css
│   │           ├── index.html
│   │           ├── js
```

```
vboxuser@niranjan: ~/devops/MyGradleapp
│   │   │   │   │   └── report.js
│   │   │   │   └── packages
│   │   │   │       └── default-package.html
│   │   ├── scripts
│   │   │   ├── MyGradleapp
│   │   │   └── MyGradleapp.bat
│   │   ├── test-results
│   │   │   └── test
│   │   │       ├── binary
│   │   │       │   ├── output.bin
│   │   │       │   ├── output.bin.idx
│   │   │       │   └── results.bin
│   │   │       └── TEST-AppTest.xml
│   │   └── tmp
│   │       ├── compileJava
│   │       ├── compileTestJava
│   │       └── jar
│   │           └── MANIFEST.MF
│   ├── build.gradle
│   ├── gradle
│   │   └── wrapper
│   │       ├── gradle-wrapper.jar
│   │       └── gradle-wrapper.properties
│   ├── gradlew
│   ├── gradlew
│   ├── gradlew.bat
│   ├── settings.gradle
│   └── src
│       ├── main
│       │   └── java
│       │       └── App.java
│       └── test
│           └── java
│               └── AppTest.java

29 directories, 26 files
vboxuser@niranjan:~/devops/MyGradleapp$
```

Display the content of build.gradle:

```
Open    build.gradle    Save
~/devops/MyGradleapp
1 /*
2 * This build file was generated by the Gradle 'init' task.
3 *
4 * This generated file contains a sample Java project to get you started.
5 * For more details take a look at the Java Quickstart chapter in the Gradle
6 * user guide available at https://docs.gradle.org/4.4.1/userguide/tutorial_java_projects.html
7 */
8
9 // Apply the java plugin to add support for Java
10 apply plugin: 'java'
11
12 // Apply the application plugin to add support for building an application
13 apply plugin: 'application'
14
15 // In this section you declare where to find the dependencies of your project
16 repositories {
17     // Use jcenter for resolving your dependencies.
18     // You can declare any Maven/Ivy/file repository here.
19     jcenter()
20 }
21
22 dependencies {
23     // This dependency is found on compile classpath of this component and consumers.
24     compile 'com.google.guava:guava:23.0'
25
26     // Use JUnit test framework
27     testCompile 'junit:junit:4.12'
28 }
29
30 // Define the main class for the application
31 mainClassName = 'App'
32 task display{
33 doLast{
34         println'hello from 6D'
35         }
36 }
```

Step 5: Execute gradle run to execute the gradle task

```
vboxuser@niranjan: ~/devops/MyGradleapp
vboxuser@niranjan:~/devops/MyGradleapp$ gradle run

> Task :run
Hello world.


BUILD SUCCESSFUL in 6s
2 actionable tasks: 1 executed, 1 up-to-date
vboxuser@niranjan:~/devops/MyGradleapp$
```

Step 6 : Execute gradle display to run an specific task specified in the build.gradle file

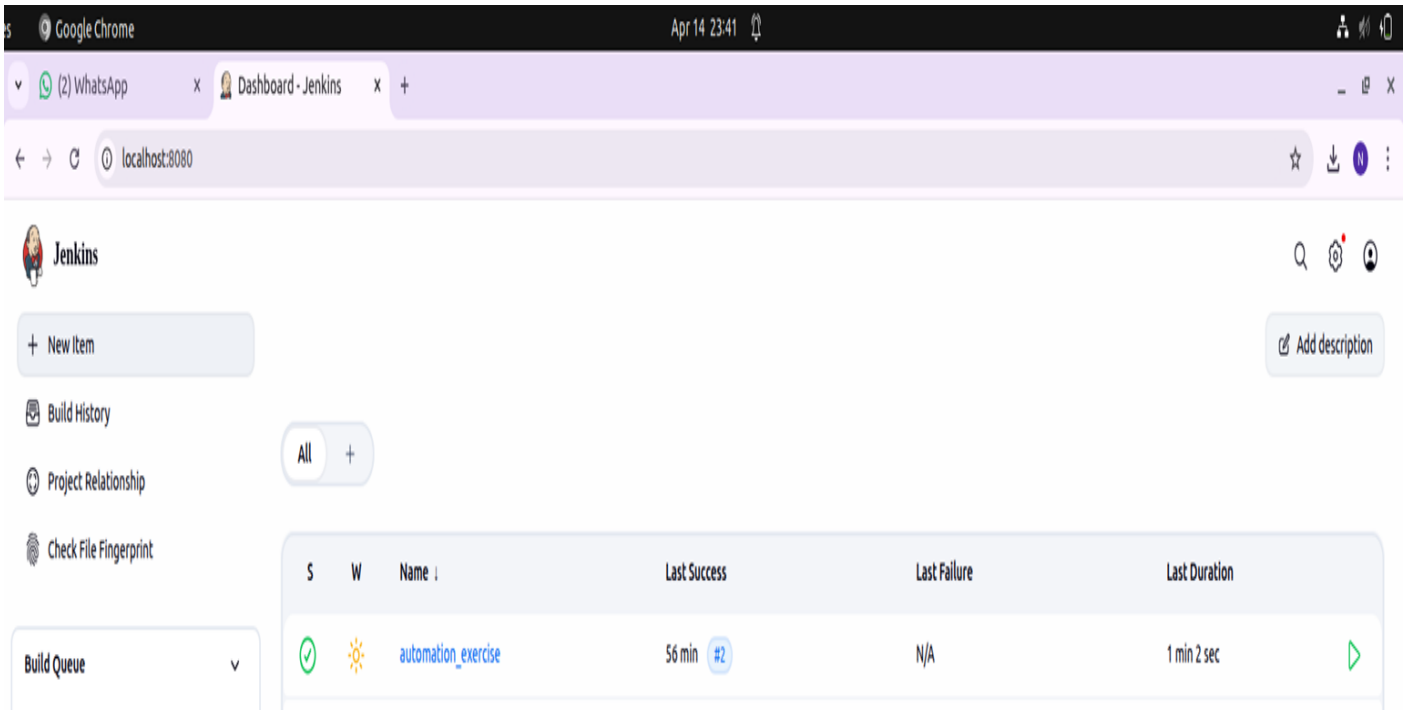
```
// we can add user defined tasks
task display{
doLast{
println 'hello from 6D'
}
}
```

Step 7:display the output after gradle display

```
vboxuser@niranjan: ~/devops/MyGradleapp
vboxuser@niranjan:~/devops/MyGradleapp$ gradle display

> Task :display
hello from 6D


BUILD SUCCESSFUL in 5s
1 actionable task: 1 executed
vboxuser@niranjan:~/devops/MyGradleapp$
```

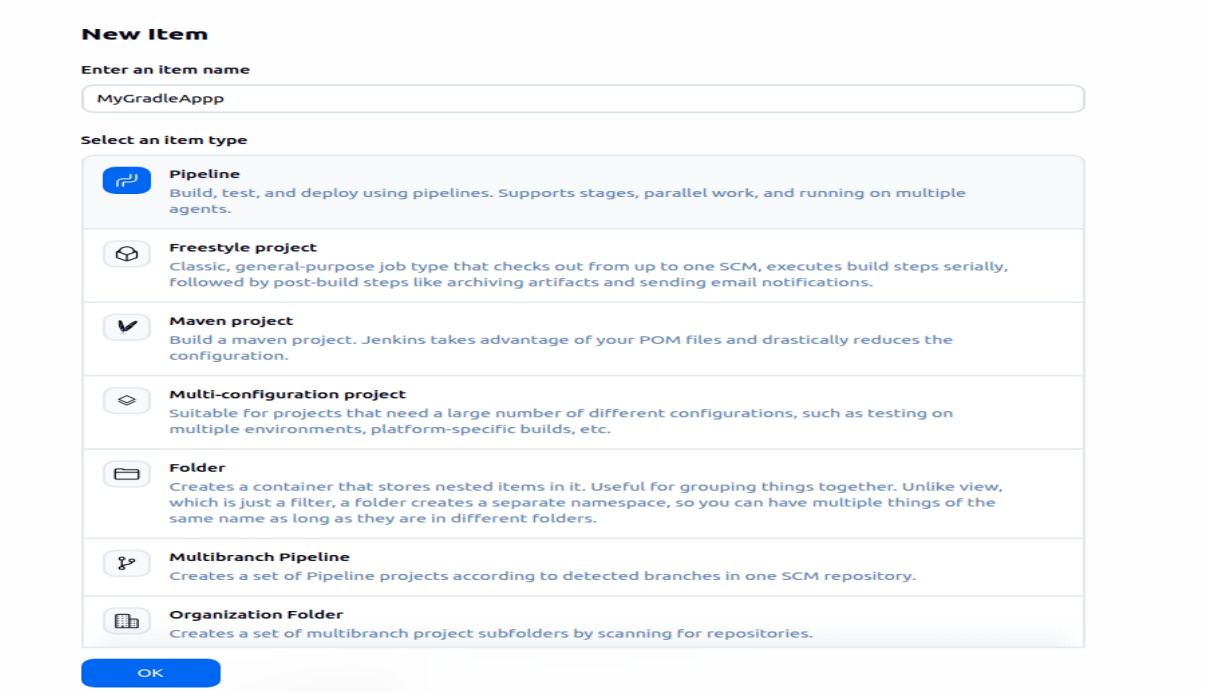
Jenkins Pipeline for MyGradleApp

Step 1: Open the Jenkins Dashboard to create a new item

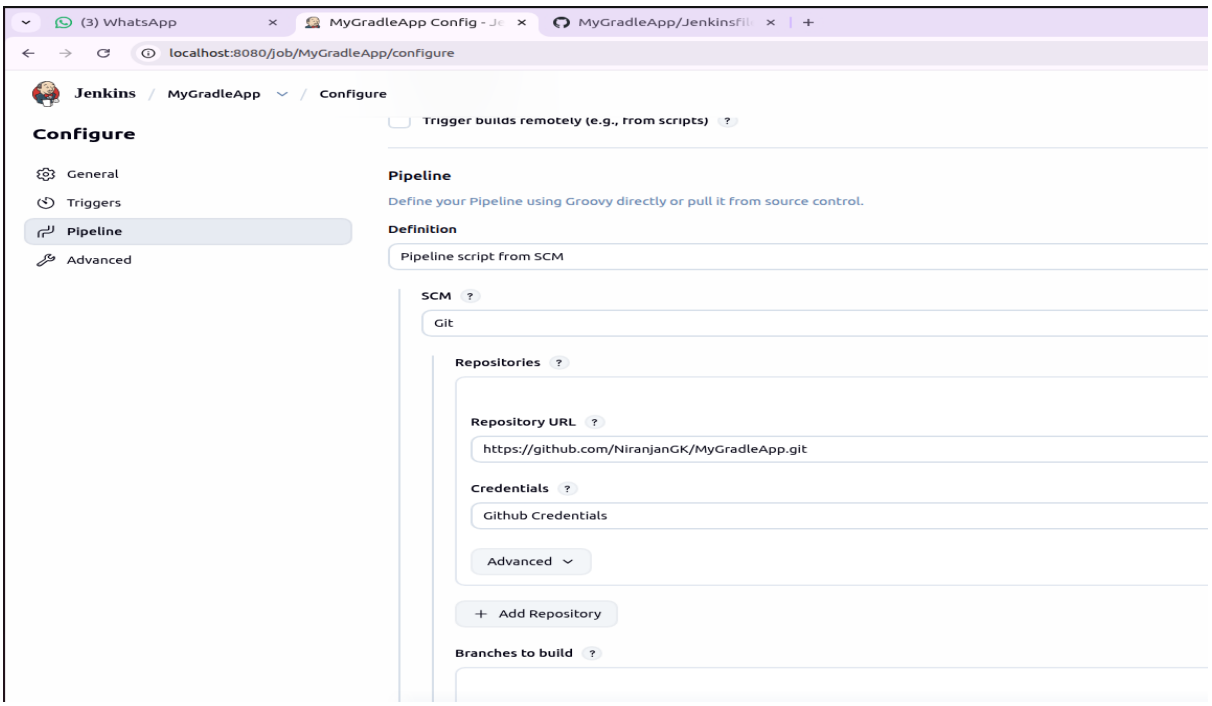


Step 2: Click on **New Item** to create a new pipeline and give the item name and select Item type as **Pipeline** and

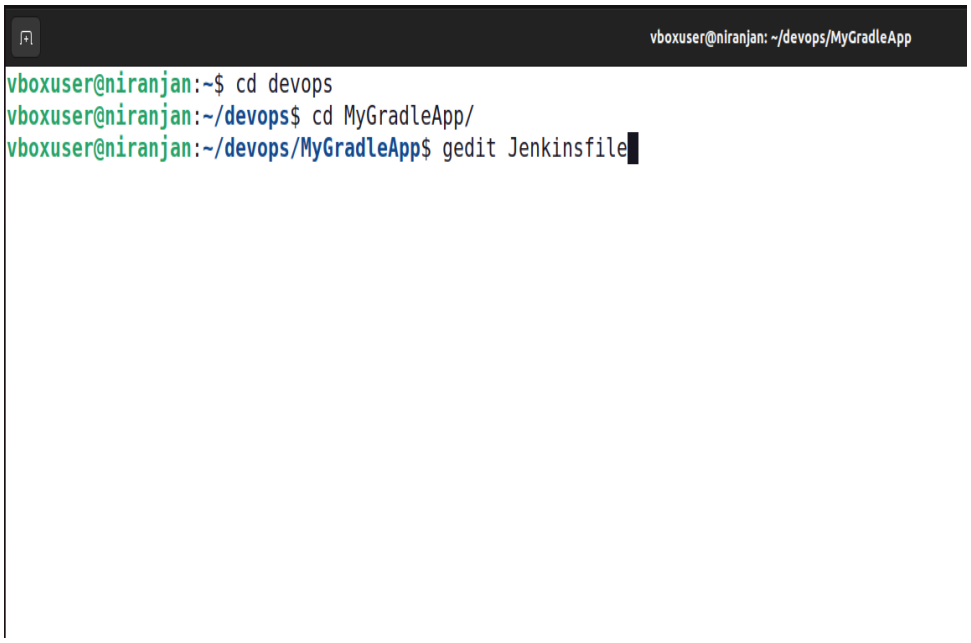
Create it



Step 3: Configure the settings set script as “Pipeline Script from SCM” and SCM as “Git”. now paste the Repository URL of the project given in GitHub and choose Credentials as “GitHub Credentials” which was created earlier.



Step 4: Now in Terminal, go to directory created for MyGradleApp and create an new file called “Jenkinsfile”



Step 5: Paste the available code inside the Jenkinsfile

```
pipeline {
    agent any  // Use any available agent
```

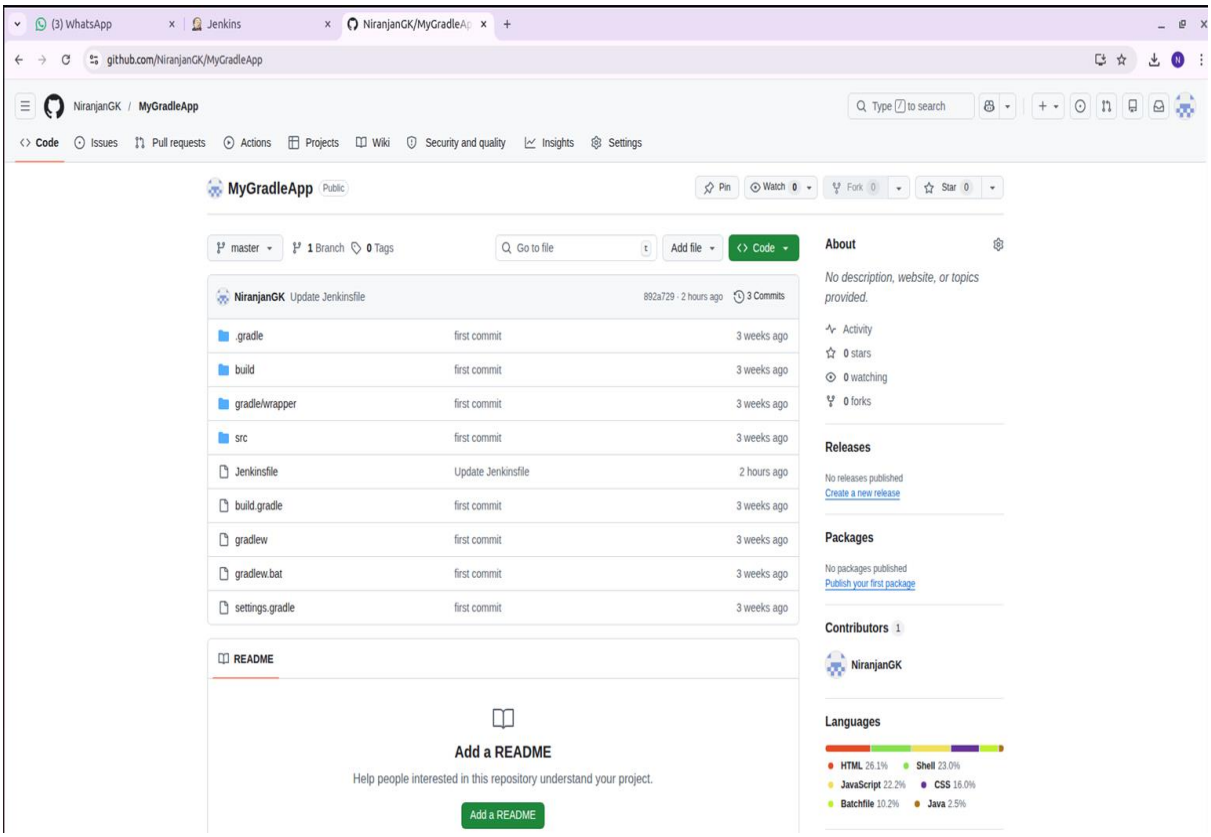
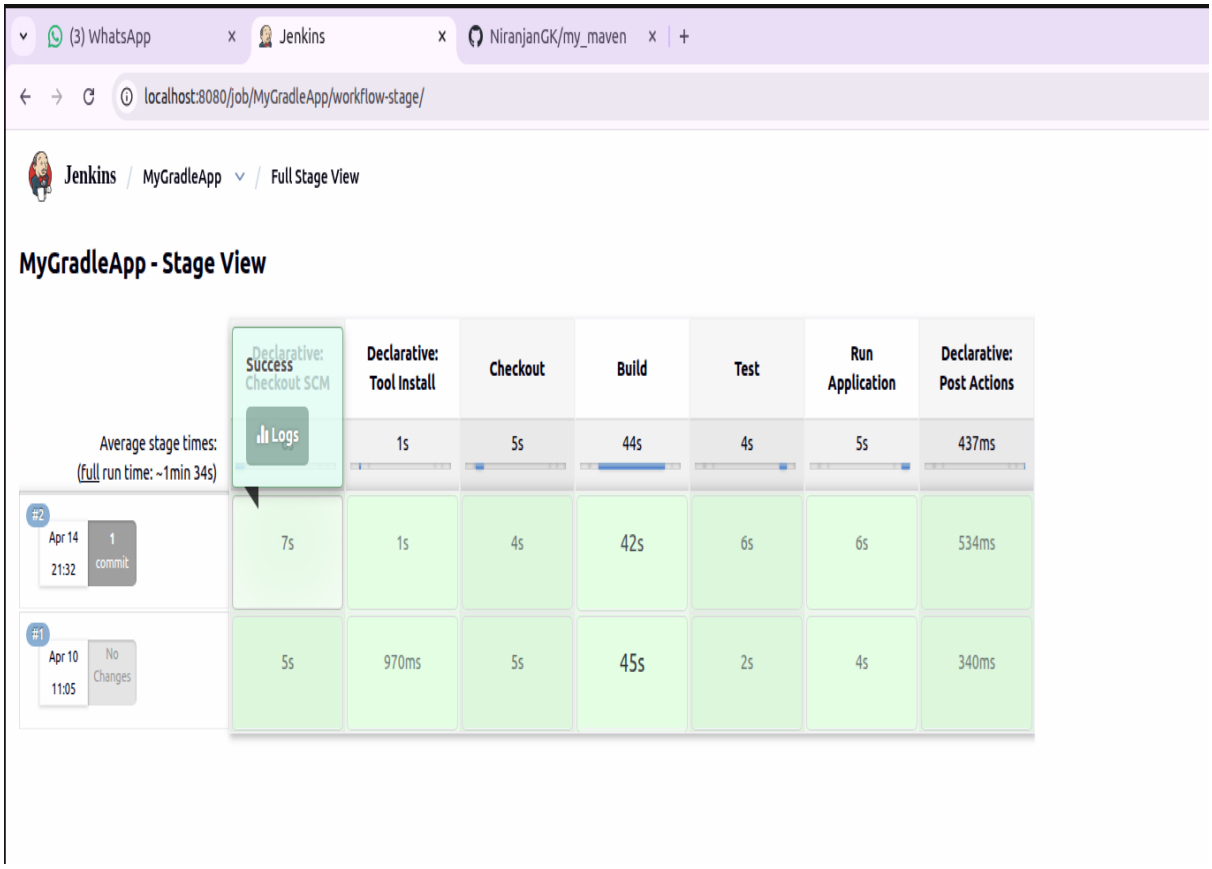
```
tools {
    gradle 'Gradle'  // Ensure this matches the name configured in Jenkins
    jdk 'JDK'
}
stages {
    stage('Checkout') {
        steps {
            git branch: 'master', url: 'https://github.com/NiranjanGK/MyGradleApp.git'
        }
    }

    stage('Build') {
        steps {
            sh 'gradle build'  // Run Gradle build
        }
    }

    stage('Test') {
        steps {
            sh 'gradle test'  // Run unit tests
        }
    }




    stage('Run Application') {
        steps {
```

```
            // Start the JAR application
            sh 'gradle run'
        }
    }



}


post {
    success {
        echo 'Build and deployment successful!'
    }
    failure {
        echo 'Build failed!'
    }
}
}
```

Step 6: make sure that the versions of sorce and target file is correct in pom.xml

```
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.8</source>
<target>1.8</target>
</configuration>
</plugin>
```

Step 7: Now we need to Push changes and commit by pushing the necessary "Jenkinsfile" to the respective Repo



Step 8: Now go to Jenkins and Build the Pipeline and thus check the Stage view for the successful execution.